

- Date: 30 September 2026
- Student-focused Generative AI learning assistant

ng A

concepts.

- Supports explanation, Q&A, quizzes, summarization and learning recommendations.
- Uses FastAPI with HTML/CSS/JavaScript and Google Gemini.

clear answer.

- Explain — simplify a difficult concept for easier understanding.
- Quiz — generate multiple-choice practice questions.
- Summary — reduce long passages into useful study notes.

- Intermediate: guided practice and connected concepts.
- Advanced: deeper topics, practice and next-step resources.
- Recommendations are generated from the student's requested topic.

- Frontend: HTML, CSS and JavaScript.
- Generative AI: Google Gemini API.
- Local AI: LaMini-Flan-T5-783M for concept explanation.

- FastAPI, Jinja2 and Uvicorn.
- Google Gemini API access through a configured API key.
- Required Python packages installed from requirements.txt.

- 2. FastAPI validates and routes the request.
- 3. Specialized AI module builds the prompt.
- 4. Gemini or the local model generates the result.
- 5. EduGenie displays the result in the web interface.

learning paths.

- LaMini-Flan-T5-783M handles concept explanation.
- AI responsibilities are separated into reusable Python modules.
- The API key is kept outside source code using environment configuration.

- The secret is not hard-coded into the repository.
- `gemini_client.py` provides a single reusable Gemini client.
- Missing configuration produces a clear runtime error.

through GEMINI_MODEL.

- The shared client sends prompts through the Google GenAI SDK.
- Modules receive clean text responses from the shared client.
- Provider configuration remains separate from business logic.

- main.py coordinates validation, routing and responses.
- Separate modules handle Q&A, explanation, quiz, summary and learning path tasks.
- JSON responses make the backend easy to test and extend.

- `gemini_client.py` — Gemini configuration and generation.
- `qna.py`, `quiz_module.py`, `summary_module.py` and `learning_path.py` — task logic.
- `templates/` and `static/` — web interface assets.

- Concept Explanation: simplified explanations using the local model.
- Quiz Generation: structured MCQs with options and answers.
- Summarization and Learning Path: concise notes and staged recommendations.

- /explain — concept explanation.
- /quiz — quiz generation.
- /summarize — long-passage summarization.
- /learn/recommendations — structured learning path.

- AI responses are normalized before returning to the frontend.
- Quiz output is parsed into structured JSON for answer checking.
- Errors are returned clearly instead of exposing implementation details.

selection and input.

- Results are displayed directly on the page.
- Quiz responses can be checked interactively.
- Responsive styling keeps the interface usable across screen sizes.

JavaScript.

- Returned content is rendered into the result area.
- Different tasks share the same interface while using different backend routes.
- The design keeps the UI simple and focused on learning.

- Install dependencies from requirements.txt.
- Run with: `uvicorn main:app --reload`.
- Open the local EduGenie page and test each major feature.

responses.

- Module tests cover quiz parsing and task behavior.
- API keys remain in environment configuration, not GitHub.
- Functional testing checks Q&A, explanation, quiz, summary and recommendations.

specialized AI learning features.

- Future extensions can include voice interaction, multilingual learning, gamification and adaptive personalization.
- Repository: [aladiyan33/EduGenie](#)
- Team ID: SWTID-2026-3580